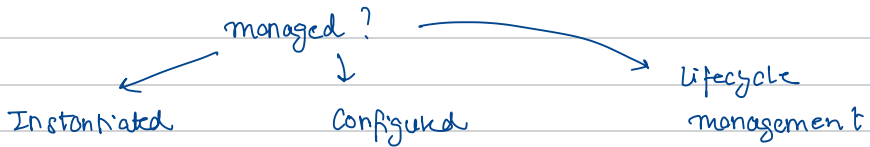


Beans in Spring Framework

What is a Bean?

- It's the basic building block of a spring application
- Simply put, these are just java objects managed by the Spring IOC container



→ we don't have to anymore create objects using the new keyword

→ If the objects of a class have to be created and used in our spring application, we will use the Spring IOC container's bean & dependency injection method.

@Component → It's an annotation (metadata) which tells the spring's IOC container that Beans be created for this class and whenever the need is present, provide it

@Component

```
class PaymentService {
```

```
    public void pay() { ... }
```

```
}
```

```
class App {
```

```
    @Autowired
```

```
    PaymentService paymentService;
```

```
    public void run() {
```

```
        paymentService.pay();
```

```
        :
```

```
    }
```

```
}
```

object / bean injected by spring's IOC container

→ no initialization by developer
using new keyword

inversion of control

@Autowired → annotation placed above an instance variable to indicate spring framework to inject Bean when needed

Alternative Method (more customised...)

→ To create beans & inject dependencies, we can use another method using @Configuration and @Bean

@Configuration → It's internally a @Component only, but it is placed on configuration classes.

↳ classes which dictate bean creation phenomenon

↳ we decide how the bean is created

↳ after that spring container takes full control over the bean's lifecycle

@Bean → Annotation placed above the constructor method which dictate the bean creation logic.

Usage ↴

@Configuration

```
class AppConfig {  
    @Bean  
    PaymentService createPaymentService (...) {  
        // some custom logic  
        :  
        return new PaymentService ();  
    }  
    :  
}
```

constructor method
can take in arg's as well

NOTE !

→ @Configuration + @Bean takes more precedence over @Component and b/w the two the premiere would dictate bean creation

Lifecycle of a Bean

→ Bean Created → Dependency Injection → Bean Initialised → Bean Used → Bean Destroyed

Lifecycle Methods

@PostConstruct → Method with this annotation are executed post bean initialisation, after all the dependencies have been injected, before the bean is actually used anywhere.

@PreDestroy → Method with this annotation is run just before the bean is about to be destroyed by the Spring IoC container. These methods are usually used for cleanup / resource releasing tasks.

Bean Scope

1. Singleton → by default, all the beans are created as singleton objects in the spring application.

→ this means everywhere the same bean is in use

@Automixd

Payment Service service1;

@Automixd

Payment Service service 2;

```
public void run () {
```

```

sout ( service 1. hashCode () );
sout ( service 2. hashCode () );

```

} same o/p

2

→ There are ways to manage Beans such that a new instance is created everytime.

Bean scopes

Scope	use-case
singleton	One instance per app, use for Services, repositories
prototype	New instance every time, use for Stateful objects, request data
request	One per HTTP request, use for Web apps only
application	One per ServletContext, use for Global config
session	One per user session, use for Shopping cart